



Data Structures and Algorithms

TP05: Linked List – Part 2

Lecturer: Ms. SEAK Leng

Name: EANG Mengly

ID: p20240009

01

```
tp5_ex1.cpp X
Week05 > lab05 > tp5_ex1.cpp > main()
1  #include <iostream>
2  #include <ctype>
3  using namespace std;
4
5  // Element structure
6  struct Element {
7      char data;
8      Element* next;
9  };
10
11 // List structure
12 struct List {
13     Element* head;
14     Element* tail;
15 };
16
17 //create an empty List
18 List* createlist() {
19     List* newList = new List();
20     newList->head = NULL;
21     newList->tail = NULL;
22     return newList;
23 }
24
25 //add an element to the end of the List
26 void addToEnd(List* List, char value) {
27     Element* newElement = new Element();
28     newElement->data = value;
29     newElement->next = NULL;
30
31     if (List->head == NULL) {
32         List->head = newElement;
33         List->tail = newElement;
34     } else {
35         List->tail->next = newElement;
36         List->tail = newElement;
37     }
38 }
39
40 //display all elements in the List
```

tp5_ex1.cpp U X

Week05 > lab05 > tp5_ex1.cpp > main()

```
39
40 //display all elements in the List
41 void displayList(List* List, string ListName) {
42     cout << "\n" << ListName << ": ";
43     Element* temp = List->head;
44     while (temp != NULL) {
45         cout << temp->data << " ";
46         temp = temp->next;
47     }
48     cout << endl;
49 }
50
51 int main() {
52     // Create two Lists
53     List* smallLetters = createList();
54     List* capitalLetters = createList();
55
56     // Add small Letters (a-z)
57     for (char ch = 'a'; ch <= 'z'; ch++) {
58         addToEnd(smallLetters, ch);
59     }
60
61     // Add capital Letters (A-Z)
62     for (char ch = 'A'; ch <= 'Z'; ch++) {
63         addToEnd(capitalLetters, ch);
64     }
65
66     // Display the Lists
67     displayList(smallLetters, "Small Letters List");
68     displayList(capitalLetters, "Capital Letters List");
69 }
```

tp5_ex1.cpp U X

Week05 > lab05 > tp5_ex1.cpp > main()

```
51 int main() {
52
53     // Clean up memory
54     Element* temp;
55     Element* current = smallLetters->head;
56     while (current != NULL) {
57         temp = current;
58         current = current->next;
59         delete temp;
60     }
61     delete smallLetters;
62
63     current = capitalLetters->head;
64     while (current != NULL) {
65         temp = current;
66         current = current->next;
67         delete temp;
68     }
69     delete capitalLetters;
70
71     return 0;
72 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

cppdbg: tp5_ex1.exe

PS N:\School\Y2-S2\data-structures-and-algorithm> & 'c:\Users\eamgm\.vscode\extensions\ms-vscode.cpptools-1.31.5-win32-x64\debugAdapters\bin\Microsoft-MIEngine-Pid-bbu2hh2z.q13' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'

Small Letters List: a b c d e f g h i j k l m n o p q r s t u v w x y z

Capital Letters List: A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

02

```
tp5_ex2.cpp X TP05-Linked list-part2.pdf
Week05 > lab05 > tp5_ex2.cpp > main()
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  //storing a person's name
6  struct Element {
7      string name;
8      Element* next;
9  };
10
11 //managing the Linked List
12 struct List {
13     Element* head;
14 };
15
16 //empty List
17 List* createlist() {
18     List* newList = new List();
19     newList->head = NULL;
20     return newList;
21 }
22
23 //add a person to the beginning of the List
24 void addPersonToBegin(List* list, string name) {
25     Element* newElement = new Element();
26     newElement->name = name;
27     newElement->next = list->head;
28     list->head = newElement;
29 }
30
31 //display all people in the List
32 void displayList(List* list) {
33     cout << "\nList of People: " << endl;
34     Element* temp = list->head;
35     int count = 1;
36     while (temp != NULL) {
37         cout << count << ". " << temp->name << endl;
38         temp = temp->next;
39         count++;
40     }
41 }
42
43 //count elements in the List
44 int countElements(List* list) {
```

tp5_ex2.cpp U X TP05-Linked list-part2.pdf

Week05 > lab05 > tp5_ex2.cpp > main()

```
43 //count elements in the list
44 int countElements(List* list) {
45     int count = 0;
46     Element* temp = list->head;
47     while (temp != NULL) {
48         count++;
49         temp = temp->next;
50     }
51     return count;
52 }
53
54 int main() {
55     List* peopleList = createList();
56
57     // Add 10 friend names to the list (demonstrating the add to begin function)
58     cout << "\nAdding 10 friends' names to the list:" << endl;
59
60     addPersonToBegin(peopleList, "Kolboth");
61     cout << "Added: Kolboth" << endl;
62     addPersonToBegin(peopleList, "Vathanak");
63     cout << "Added: Vathanak" << endl;
64     addPersonToBegin(peopleList, "Chet");
65     cout << "Added: Chet" << endl;
66     addPersonToBegin(peopleList, "Both");
67     cout << "Added: Both" << endl;
68     addPersonToBegin(peopleList, "Nana");
69     cout << "Added: Nana" << endl;
70     addPersonToBegin(peopleList, "Caro");
71     cout << "Added: Caro" << endl;
72     addPersonToBegin(peopleList, "Nika");
73     cout << "Added: Nika" << endl;
74     addPersonToBegin(peopleList, "Sak");
75     cout << "Added: Sak" << endl;
76     addPersonToBegin(peopleList, "Rithysak");
77     cout << "Added: Rithysak" << endl;
78     addPersonToBegin(peopleList, "Heang");
79     cout << "Added: Heang" << endl;
80
81     // Display all people in the list
82     displayList(peopleList);
83
84     // Display the count
85     cout << "\nTotal number of people in the list: " << countElements(peopleList) << endl;
```

tp5_ex2.cpp TP05-Linked list-part2.pdf

Week05 > lab05 > tp5_ex2.cpp > main()

```
54 int main() {
55     // Display the count
56     cout << "\nTotal number of people in the list: " << countElements(peopleList) << endl;
57
58     // Clean up memory
59     Element* temp;
60     Element* current = peopleList->head;
61     while (current != NULL) {
62         temp = current;
63         current = current->next;
64         delete temp;
65     }
66     delete peopleList;
67
68     return 0;
69 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

cppdbg: tp5_ex2.exe

● PS N:\School\Y2-S2\data-structures-and-algorithm> & 'c:\Users\eamgm\.vscode\extensions\ms-vscode.cpptools-1.31.5-win32-x64\debugAdapters\bin\cher.exe' '--stdin=Microsoft-MIEngine-In-taig1yns.ffu' '--stdout=Microsoft-MIEngine-Out-ycceg5s1.lhn' '--stderr=Microsoft-MIEngine-Error-hzqwdl Microsoft-MIEngine-Pid-q4amop05.2i0' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'

==== Exercise 2: Storing Names in a Linked List (Add to Begin) ====

Adding 10 friends' names to the list:

Added: Kolboth
Added: Vathanak
Added: Chet
Added: Both
Added: Nana
Added: Caro
Added: Nika
Added: Sak
Added: Rithysak
Added: Heang

List of People:

1. Heang
2. Rithysak
3. Sak
4. Nika
5. Caro
6. Nana
7. Both
8. Chet
9. Vathanak
10. Kolboth

Total number of people in the list: 10

○ PS N:\School\Y2-S2\data-structures-and-algorithm>

03

```
G: tp5_ex3.cpp U X
Week05 > lab05 > G: tp5_ex3.cpp > main()
1  #include <iostream>
2  #include <string>
3  #include <iomanip>
4  using namespace std;
5
6  // Product structure
7  struct Product {
8      int productId;
9      string name;
10     double price;
11 };
12
13 // Element structure for Linked List node
14 struct Element {
15     Product product;
16     Element* next;
17 };
18
19 // List structure for managing the Linked List
20 struct ListProduct {
21     Element* head;
22 };
23
24 // create an empty List
25 ListProduct* createEmptyList() {
26     ListProduct* newList = new ListProduct();
27     newList->head = NULL;
28     return newList;
29 }
30
31 // add product to the beginning of the List (when price < 50)
32 void addProductToBegin(ListProduct* List, int id, string name, double price) {
33     if (price >= 50) {
34         cout << "Error: Price must be less than $50 to add at the beginning!" << endl;
35         return;
36     }
37
38     Element* newElement = new Element();
39     newElement->product.productId = id;
40     newElement->product.name = name;
41     newElement->product.price = price;
42     newElement->next = List->head;
43     List->head = newElement;
44
45     cout << "Product added to begin: " << name << endl;
46 }
47
```

tp5_ex3.cpp U X

Week05 > lab05 > tp5_ex3.cpp > main()

```
48 // add product to the end of the list (when price >= 50)
49 void addProductToEnd(ListProduct* List, int id, string name, double price) {
50     if (price < 50) {
51         cout << "Error: Price must be at least $50 to add at the end!" << endl;
52         return;
53     }
54
55     Element* newElement = new Element();
56     newElement->product.productId = id;
57     newElement->product.name = name;
58     newElement->product.price = price;
59     newElement->next = NULL;
60
61     if (List->head == NULL) {
62         List->head = newElement;
63     } else {
64         Element* temp = List->head;
65         while (temp->next != NULL) {
66             temp = temp->next;
67         }
68         temp->next = newElement;
69     }
70
71     cout << "Product added to end: " << name << endl;
72 }
73
74 // display all products in the list
75 void display(ListProduct* List) {
76     cout << "\n===== Product List =====" << endl;
77     cout << left << setw(5) << "ID" << setw(20) << "Name" << setw(10) << "Price" << endl;
78     cout << "===== " << endl;
79
80     Element* temp = List->head;
81     if (temp == NULL) {
82         cout << "The list is empty!" << endl;
83     } else {
84         while (temp != NULL) {
85             cout << left << setw(5) << temp->product.productId
86                 << setw(20) << temp->product.name
87                 << "$" << fixed << setprecision(2) << temp->product.price << endl;
88             temp = temp->next;
89         }
90     }
91     cout << "===== " << endl;
92 }
93
```


tp5_ex3.cpp

Week05 > lab05 > tp5_ex3.cpp > ...

```
94 int main() {
95     ListProduct* productList = createEmptyList();
96
97     // Test the program by adding various products
98     cout << "\nAdding products to the list:" << endl;
99
100    // Products with price < 50 (add to begin)
101    addProductToBegin(productList, 101, "Pencil", 2.50);
102    addProductToBegin(productList, 102, "Notebook", 5.75);
103    addProductToBegin(productList, 103, "Eraser", 1.25);
104
105    // Products with price >= 50 (add to end)
106    addProductToEnd(productList, 201, "Monitor", 299.99);
107    addProductToEnd(productList, 202, "Keyboard", 79.50);
108    addProductToEnd(productList, 203, "Mouse", 49.99);
109
110    // Display all products
111    display(productList);
112
113    // Clean up memory
114    Element* temp;
115    Element* current = productList->head;
116    while (current != NULL) {
117        temp = current;
118        current = current->next;
119        delete temp;
120    }
121    delete productList;
122
123    return 0;
124 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

cppdbg: tp5_ex3.exe

PS N:\School\Y2-S2\data-structures-and-algorithm> & 'c:\Users\eamgm\.vscode\extensions\ms-vscode.cpptools-1.31.5-win32-x64\debugAdapters\bin\chex.exe' '--stdin=Microsoft-MIEngine-In-c2roemj.3nh' '--stdout=Microsoft-MIEngine-Out-tomsupy.55u' '--stderr=Microsoft-MIEngine-Error-itc Microsoft-MIEngine-Pid-0namrkw.2du' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'

Product added to end: Monitor

Product added to end: Keyboard

Error: Price must be at least \$50 to add at the end!

===== Product List =====

ID	Name	Price
103	Eraser	\$1.25
102	Notebook	\$5.75
101	Pencil	\$2.50
201	Monitor	\$299.99
202	Keyboard	\$79.50

=====

PS N:\School\Y2-S2\data-structures-and-algorithm>

04

```
tp5_ex4.cpp U X
Week05 > lab05 > tp5_ex4.cpp > main()
1  #include <iostream>
2  #include <string>
3  #include <iomanip>
4  using namespace std;
5
6  // Product structure
7  struct Product {
8      int productId;
9      string name;
10     double price;
11 };
12
13 // Element structure for Linked List node
14 struct Element {
15     Product product;
16     Element* next;
17 };
18
19 // List structure for managing the Linked List
20 struct ListProduct {
21     Element* head;
22 };
23
24 // create an empty List
25 ListProduct* createEmptyList() {
26     ListProduct* newList = new ListProduct();
27     newList->head = NULL;
28     return newList;
29 }
30
31 // add product to the beginning of the List (when price < 50)
32 void addProductToBegin(ListProduct* list, int id, string name, double price) {
33     if (price >= 50) {
34         cout << "Error: Price must be less than $50 to add at the beginning!" << endl;
35         return;
36     }
37
38     Element* newElement = new Element();
39     newElement->product.productId = id;
40     newElement->product.name = name;
41     newElement->product.price = price;
42     newElement->next = list->head;
43     list->head = newElement;
44
45     cout << "Product added to begin: " << name << endl;
46 }
47
```

tp5_ex4.cpp U X

Week05 > lab05 > tp5_ex4.cpp > main()

```
47
48 // add product to the end of the List (when price >= 50)
49 void addProductToEnd(ListProduct* List, int id, string name, double price) {
50     if (price < 50) {
51         cout << "Error: Price must be at least $50 to add at the end!" << endl;
52         return;
53     }
54
55     Element* newElement = new Element();
56     newElement->product.productId = id;
57     newElement->product.name = name;
58     newElement->product.price = price;
59     newElement->next = NULL;
60
61     if (List->head == NULL) {
62         List->head = newElement;
63     } else {
64         Element* temp = List->head;
65         while (temp->next != NULL) {
66             temp = temp->next;
67         }
68         temp->next = newElement;
69     }
70
71     cout << "Product added to end: " << name << endl;
72 }
73
74 // display all products in the List
75 void display(ListProduct* List) {
76     cout << "\n===== Product List =====" << endl;
77     cout << left << setw(5) << "ID" << setw(20) << "Name" << setw(10) << "Price" << endl;
78     cout << "===== " << endl;
79
80     Element* temp = List->head;
81     if (temp == NULL) {
82         cout << "The list is empty!" << endl;
83     } else {
84         while (temp != NULL) {
85             cout << left << setw(5) << temp->product.productId
86                 << setw(20) << temp->product.name
87                 << "$" << fixed << setprecision(2) << temp->product.price << endl;
88             temp = temp->next;
89         }
90     }
91     cout << "===== " << endl;
92 }
93
```

tp5_ex4.cpp U X

Week05 > lab05 > tp5_ex4.cpp > main()

```
94 // search and display product by name
95 void searchByName(ListProduct* List, string searchName) {
96     cout << "\n--- Search Results by Name: '" << searchName << "' ---" << endl;
97     Element* temp = List->head;
98     bool found = false;
99
100     while (temp != NULL) {
101         if (temp->product.name == searchName) {
102             cout << left << setw(5) << "ID: " << temp->product.productId << endl;
103             cout << left << setw(5) << "Name: " << temp->product.name << endl;
104             cout << left << setw(5) << "Price: $" << fixed << setprecision(2) << temp->product.price << endl;
105             found = true;
106         }
107         temp = temp->next;
108     }
109
110     if (!found) {
111         cout << "No product found with name: " << searchName << endl;
112     }
113 }
114
115 // search and display products by price
116 void searchByPrice(ListProduct* List, double price) {
117     cout << "\n===== Search Results by Price (p = $" << fixed << setprecision(2) << price << ") =====" << endl;
118
119     Element* temp = List->head;
120     bool foundBelow = false, foundAbove = false;
121
122     // Section 1: Products with price Lower than p
123     cout << "\nSection 1: Products with price LOWER than $" << price << endl;
124     cout << left << setw(5) << "ID" << setw(20) << "Name" << setw(10) << "Price" << endl;
125     cout << "-----" << endl;
126
127     temp = List->head;
128     while (temp != NULL) {
129         if (temp->product.price < price) {
130             cout << left << setw(5) << temp->product.productId
131                 << setw(20) << temp->product.name
132                 << "$" << fixed << setprecision(2) << temp->product.price << endl;
133             foundBelow = true;
134         }
135         temp = temp->next;
136     }
137     if (!foundBelow) {
138         cout << "No products found." << endl;
139     }
140 }
```

G+ tp5_ex4.cpp U X

Week05 > lab05 > G+ tp5_ex4.cpp > main()

```
116 void searchByPrice(ListProduct* List, double price) {
141     // Section 2: Products with price more than p
142     cout << "\nSection 2: Products with price MORE than $" << price << endl;
143     cout << left << setw(5) << "ID" << setw(20) << "Name" << setw(10) << "Price" << endl;
144     cout << "-----" << endl;
145
146     temp = List->head;
147     while (temp != NULL) {
148         if (temp->product.price > price) {
149             cout << left << setw(5) << temp->product.productId
150                 << setw(20) << temp->product.name
151                 << "$" << fixed << setprecision(2) << temp->product.price << endl;
152             foundAbove = true;
153         }
154         temp = temp->next;
155     }
156     if (!foundAbove) {
157         cout << "No products found." << endl;
158     }
159 }
160
161 // Function to update product information by product ID
162 void updateByProductId(ListProduct* List, int id, string newName, double newPrice) {
163     Element* temp = List->head;
164     bool found = false;
165
166     while (temp != NULL) {
167         if (temp->product.productId == id) {
168             cout << "\nUpdating Product ID " << id << ":" << endl;
169             cout << "Old Name: " << temp->product.name << " -> New Name: " << newName << endl;
170             cout << "Old Price: $" << fixed << setprecision(2) << temp->product.price
171                 << " -> New Price: $" << newPrice << endl;
172
173             temp->product.name = newName;
174             temp->product.price = newPrice;
175             found = true;
176             break;
177         }
178         temp = temp->next;
179     }
180
181     if (!found) {
182         cout << "Product with ID " << id << " not found!" << endl;
183     }
184 }
185
```

C: tp5_ex4.cpp U X

Week05 > lab05 > C: tp5_ex4.cpp > main()

```
186 int main() {
187     ListProduct* productList = createEmptyList();
188
189     // Add various products to the list
190     cout << "\nAdding products to the list:" << endl;
191
192     // Products with price < 50 (add to begin)
193     addProductToBegin(productList, 101, "Pencil", 2.50);
194     addProductToBegin(productList, 102, "Notebook", 5.75);
195     addProductToBegin(productList, 103, "Eraser", 1.25);
196     addProductToBegin(productList, 104, "Pen", 3.99);
197
198     // Products with price >= 50 (add to end)
199     addProductToEnd(productList, 201, "Monitor", 299.99);
200     addProductToEnd(productList, 202, "Keyboard", 79.50);
201     addProductToEnd(productList, 203, "Mouse", 49.99);
202     addProductToEnd(productList, 204, "Headphones", 89.99);
203
204     // Display all products
205     display(productList);
206
207     // Test search by name
208     searchByName(productList, "Monitor");
209     searchByName(productList, "Keyboard");
210
211     // Test search by price
212     searchByPrice(productList, 50.0);
213
214     // Test update function
215     updateByProductId(productList, 201, "Dell Monitor", 349.99);
216     updateByProductId(productList, 102, "Premium Notebook", 12.99);
217     updateByProductId(productList, 999, "Non-existent", 100.0);
218
219     // Display updated list
220     display(productList);
221
222     // Clean up memory
223     Element* temp;
224     Element* current = productList->head;
225     while (current != NULL) {
226         temp = current;
227         current = current->next;
228         delete temp;
229     }
230     delete productList;
231
232     return 0;
233 }
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
PS N:\School\Y2-S2\data-structures-and-algorithm> & 'c:\Users\eamgm\.vscode\extensions\ms-vscode.cpptools-1.31.5-win32-x64\debugAdapter:
cher.exe' '--stdin=Microsoft-MIEngine-In-sld4npaq.yjc' '--stdout=Microsoft-MIEngine-Out-pskapl2u.e1u' '--stderr=Microsoft-MIEngine-Error
Microsoft-MIEngine-Pid-jpabm3jk.dln' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Product added to end: Monitor
Product added to end: Keyboard
Error: Price must be at least $50 to add at the end!
Product added to end: Headphones

===== Product List =====
ID   Name           Price
=====
104   Pen             $3.99
103   Eraser          $1.25
102   Notebook        $5.75
101   Pencil          $2.50
201   Monitor         $299.99
202   Keyboard        $79.50
204   Headphones      $89.99
=====

--- Search Results by Name: 'Monitor' ---
ID: 201
Name: Monitor
Price: $299.99

--- Search Results by Name: 'Keyboard' ---
ID: 202
Name: Keyboard
Price: $79.50

===== Search Results by Price (p = $50.00) =====

Section 1: Products with price LOWER than $50.00
ID   Name           Price
-----
104   Pen             $3.99
103   Eraser          $1.25
102   Notebook        $5.75
101   Pencil          $2.50

Section 2: Products with price MORE than $50.00
ID   Name           Price
-----
201   Monitor         $299.99
202   Keyboard        $79.50
204   Headphones      $89.99

Updating Product ID 201:
Old Name: Monitor -> New Name: Dell Monitor
Old Price: $299.99 -> New Price: $349.99

Updating Product ID 102:
Old Name: Notebook -> New Name: Premium Notebook
Old Price: $5.75 -> New Price: $12.99
Product with ID 999 not found!
```

Github Link:

<https://github.com/mengly-eg/data-structures-and-algorithm.git>